Welcome to Mars. Please don’t die

Running late is our cardio

1835365 - VLAD TURNO
1883008 - GABRIELE INCARBONE
1950722 - DAVIDE TIMPERI
1986026 - FEDERICO MENDIOLA

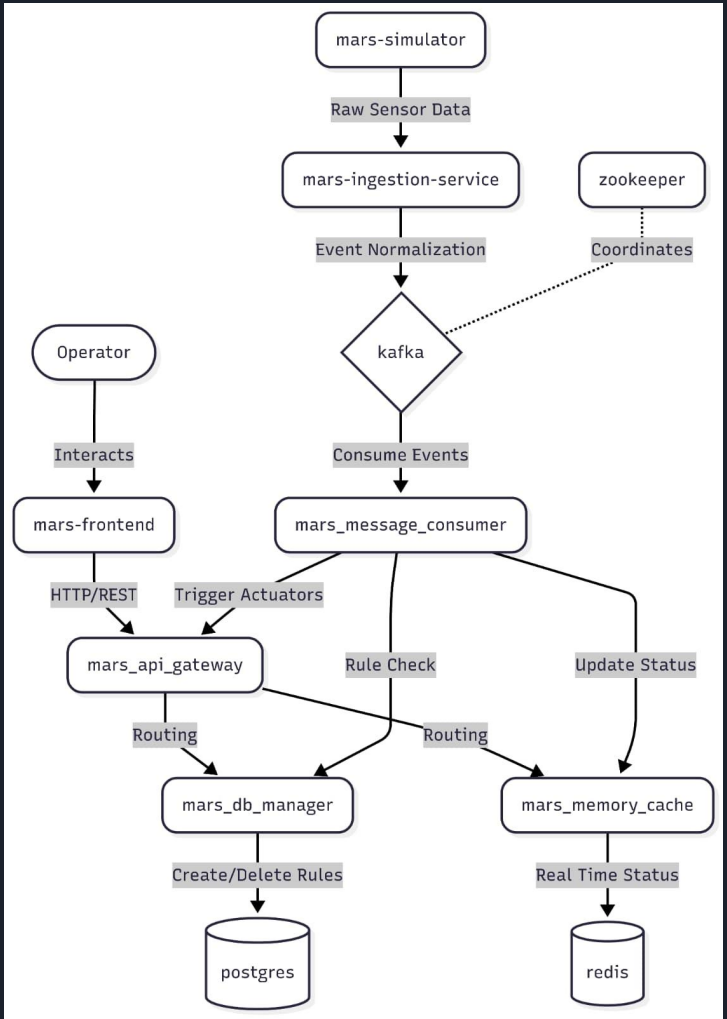
Welcome to Mars.

Please don’t die

Running late is our cardio

1835365 - VLAD TURNO
1883008 - GABRIELE INCARBONE
1950722 - DAVIDE TIMPERI
1986026 - FEDERICO MENDIOLA

system architecture diagram



SYSTEM ARCHITECTURE DIAGRAM

Mars Ingestion Service : Steaming & Polling

Role: Primary bridge between Mars IoT sensors and downstream processing

Tech: Spring Boot , Java HTTP Client , Jackson JSON

- MarsScheduledIngestionService (API Polling):
  - @Scheduled HTTP polling every 5 seconds
  - 8 IoT sensors (Temperature, Humidity, CO2, pH, Water Level, Pressure, Air Quality)
  - Synchronous health checks → sensor polling → JSON mapping → Kafka publish
- MarsTelemetryStreamService (Server-Sent Events Streaming):
  - Approach: Real-time SSE streaming with persistent HTTP connections
  - 7 telemetry streams (Solar Array, Power Bus, Power Consumption, Radiation, Life Support, Thermal Loop, Airlock)
  - Mechanism: @PostConstruct startup → async stream connections → continuous event reading → real-time Kafka publish

Mars Ingestion Service : Steaming & Polling

Role: Primary bridge between Mars IoT sensors and downstream processing

Tech: Spring Boot , Java HTTP Client , Jackson JSON

MarsScheduledIngestionService
(API Polling)

8 IoT sensors
(Temperature, Humidity, CO2, pH, Water Level, Pressure, Air Quality)

→ Synchronous health checks → sensor polling → JSON mapping → Kafka publish

@Scheduled HTTP polling every 5 seconds

MarsTelemetryStreamService
(Server-Sent Events Streaming)

7 telemetry streams
(Solar Array, Power Bus, Power Consumption, Radiation, Life Support, Thermal Loop, Airlock)

→ @PostConstruct startup → async stream connections → continuous event reading → real-time Kafka publish

Approach: Real-time SSE streaming with persistent HTTP connections

Mars Ingestion Service : Normalization

Role: Transform raw sensor data into a unified event schema

Tech: Java + Object mapper

- Its fundamental purpose is to solve the heterogeneity problem - we receive data from 10+ different sensor types with varying formats. The Normalizer acts as a universal translator, converting these formats into a single standardized Event schema with consistent fields: timestamp, sensor name, value, unit, and status.
- It's intelligent about transformations too - simple scalar sensors map 1:1, but complex sensors like particulate monitors expand into multiple events (PM1, PM2.5, PM10). The chemistry sensor splits into individual metric events, and water level sensors generate both percentage and liter readings.
- This normalization layer is what enables downstream services to consume all Mars habitat data through a single, predictable interface.

Mars Ingestion Service : Normalization

Role: Transform raw sensor data into a unified event schema

Tech: Java + Object mapper

Role: Transform raw sensor data into a unified event schema

Heterogeneous Raw Sensor Data (10+ Types)

Normalizer (Universal Translator)

Intelligent Transformations: Maps simple scalars 1:1; expands complex sensors (e.g., Particulate Monitors) into multiple events (PM1, PM2.5, PM10); splits Chemistry & Water Level sensors into individual metric events (percentage, liters).

Standardized Event Schema

Downstream Consumption (Single Interface)

Enables downstream services to consume all Mars habitat data through a single, predictable interface.

Mars Ingestion Service : Kafka Producer

Role: Publishes normalized Mars telemetry data from the ingestion services to Kafka topics for downstream consumption

Tech: CompletableFuture ,Apache Kafka

- The now normalized events are now ready to be published into our topic `mars-telemetry-events`
- `The events are published asynchronously` , this prevents Kafka I/O from blocking the 5-second polling cycles - the ingestion services continue collecting data while Kafka processes messages in the background
- We use sensor names as message keys to ensure consistent partitioning - all readings from the same sensor go to the same partition, maintaining chronological order.

Mars Ingestion Service : Kafka Producer

Role: Transform raw sensor data into a unified event schema

Tech: Java + Object mapper

Role: Publishes normalized Mars telemetry data from the ingestion services to Kafka

Normalized Data from Ingestion Services

Events are published asynchronously, preventing I/O from blocking the 5-second polling cycles - ingestion continues collecting data while Kafka processes in background.

Sensor names used as message keys for consistent partitioning - all readings from the same sensor sensor go to the same partition, maintaining chronological order.

Mars Message Consumer: The Event Processor

Role: The background worker handling high-throughput telemetry data.
Tech: Python, confluent-kafka

- Listens to the mars-telemetry-events Kafka topic in an infinite loop. Kafka acts as a shock-absorber, preventing system crashes during data spikes.
- For every reading, it queries the DB Manager (PostgreSQL) to check if the new value triggers any Automation Rules (e.g., "if temp > 35, turn on fan").
- If rules trigger, it commands the API Gateway to physically toggle the actuator. It then saves the new reading to the Cache and tells the Gateway to broadcast it.

Mars Message Consumer: The Event Processor

Role: The background worker handling high-throughput telemetry data.
Tech: Python, confluent-kafka

- Listens to the **mars-telemetry-events Kafka** topic in an infinite loop. Kafka acts as a shock-absorber, preventing system crashes during data spikes.

- For every reading, it queries the DB Manager (PostgreSQL) to check if the new value triggers any Automation Rules (e.g., “if temp > 35, turn on fan”).

- If rules trigger, it commands the API Gateway to physically toggle the actuator. It then saves the new reading to the Cache and tells the Gateway to broadcast it.

Mars Memory Cache: The Real-Time State

Role: The ultra-fast holder of the facility's current status.
Tech: Python, FastAPI, Redis

- Redis stores data entirely in RAM. It eliminates slow disk reads when checking the last known value of a sensor.
- It only stores the absolute latest state of every sensor. (e.g., it overwrites temp: 22 with temp: 23).
- If rules trigger, it commands the API Gateway to physically toggle the actuator. When a user opens the dashboard, the Gateway fetches this entire cache instantly. This makes the UI feel infinitely responsive without hammering a traditional database.

Mars Memory Cache: The Real-Time State

Role & Tech

Role: The ultra-fast holder of the facility's current status.

Tech: Python, FastAPI, Redis

- Redis stores data entirely in RAM. It eliminates slow disk reads when checking the last known value of a sensor.
- It only stores the absolute latest state of every sensor. (e.g., it overwrites temp: 22 with temp: 23).
- If rules trigger, it commands the API Gateway to physically toggle the actuator. When a user opens the dashboard, the Gateway fetches this entire cache instantly. This makes the UI feel infinitely responsive without hammering a traditional database.

Relational DB implementation with Postgres 15

- Rule schema designed to natively map the automation rules expressed by IF-THEN-ELSE conditions

Table: | sensor_name | operator | threshold_value | actuator_device | actuator_state |

Row: |temperature | > | 30 °C | cooling fan | ON |

- Dedicated microservice (with Python/FastAPI) to separate database logic from the rest of the system
- Secure endpoints exposed for data operations

Relational DB implementation with Postgres 15

Rule Schema & Database Table

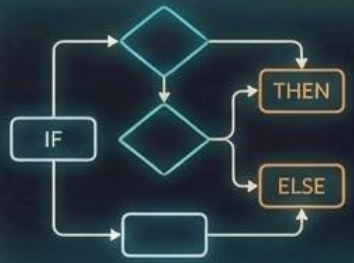


Table:

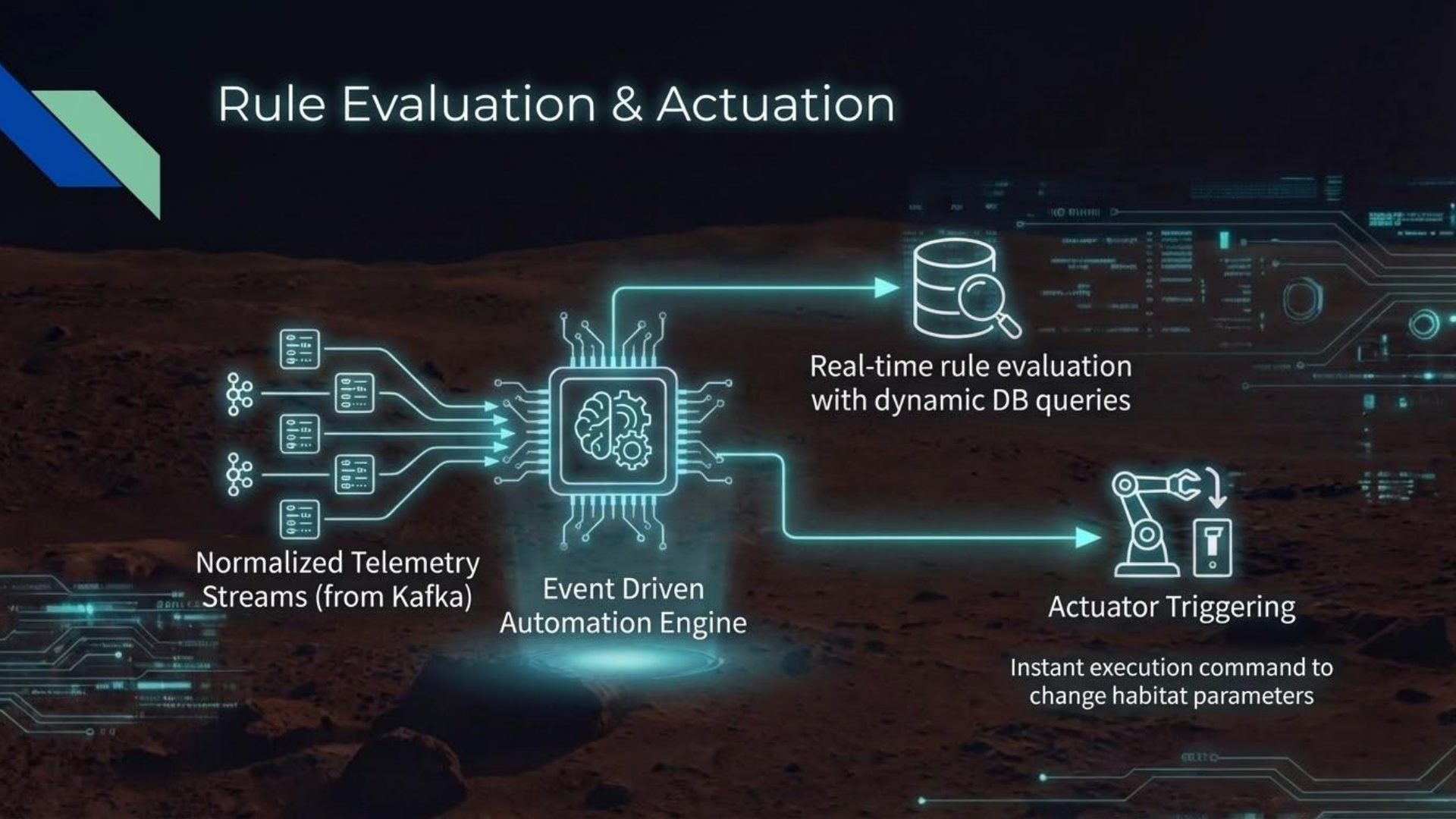
sensor_name	operator	threshold_value	actuator_device	actuator_state
temperature	>	30 °C	cooling fan	ON

- Dedicated microservice (with Python/FastAPI) to separate database logic from the rest of the system
- Secure endpoints exposed for data operations

Rule Evaluation & Actuation

- Event driven automation engine
- Continuous and asynchronous listening to normalized telemetry streams arriving from Kafka
- Real time rule evaluation with dynamic db queries
- Actuator triggering: When a threshold is breached, the consumer instantly sends an execution command to the API Gateway to change the habitat's parameters.

Rule Evaluation & Actuation



Middleware & API Gateway

- `mars_api_gateway` as the sole middleware routing traffic between the React Dashboard and internal services

- Implemented routing functions for creating (POST), reading (GET), and deleting (DELETE) rules

- Core services protected: the gateway is the only endpoint exposed!

Middleware & API Gateway

Core services protected: the gateway is the only endpoint exposed!

Mars API Gateway: The Frontend BFF

Role: The central hub ("Backend-for-Frontend") connecting React to the microservices.
Tech: Python, FastAPI, WebSockets

- Consolidates access to the DB, Cache, and Simulator under one localhost:8000 roof, solving difficult browser CORS issues.
- Maintains open WebSockets with all active users. When the Consumer broadcasts a new event, the Gateway pushes it instantly to the React app.
- Proxies frontend requests (like creating or deleting automation rules) to the isolated DB Manager, enforcing strict separation of concerns.

Mars API Gateway: The Frontend BFF

Role & Tech

Role: The central hub ("Backend-for-Frontend") connecting React to the microservices.
Tech: Python, FastAPI, WebSockets

Consolidates access to the DB, Cache, and Simulator under one localhost:8000 roof, solving difficult browser CORS issues.

Maintains open WebSockets with all active users. When the Consumer broadcasts a new event, the Gateway pushes it instantly to the React app.

Proxies frontend requests (like creating or deleting automation rules) to the isolated DB Manager, enforcing strict separation of concerns.

MARS_OS Command Center: Frontend Architecture

- **Objective**: develop an operational Head-Up Display (HUD) for real-time monitoring and control of the Mars base.
- **Key Requirements**: instant reactivity, zero page reloads (Single Page Application), and complete decoupling from the backend services.

MARS_OS Command Center: Frontend Architecture

Objective

Objective:
develop an operational Head-Up Display (HUD) for real-time monitoring and control of the Mars base.

Key Requirements

- instant reactivity
- zero page reloads (Single Page Application)
- complete decoupling from the backend services.

Data Flow: From Sensor to Screen

- **No Polling, Just Push:** leveraging WebSockets for a persistent, bidirectional connection with the API Gateway.
- **Zero Latency:** Kafka-processed events are pushed instantly to the React client.
- **Dynamic Data Mapping:** Runtime formatting function to translate raw sensor paths into clean UI labels, preserving original data integrity for the backend.

Data Flow: From Sensor to Screen

No Polling, Just Push

leveraging WebSockets for a persistent, bidirectional connection with the API Gateway.

Zero Latency

Kafka-processed events are pushed instantly to the React client.

Dynamic Data Mapping

Runtime formatting function to translate raw sensor paths into clean UI labels, preserving original data integrity for the backend.

Optimizing User Experience (UX)

- **Live Historical Charting (Recharts):** sliding time window (last 20 samples) for Greenhouse Temperature, updating dynamically on every WebSocket event.
- **Optimistic UI for Actuators:** immediate visual feedback upon operator click.
  - Asynchronous REST API (POST) execution in the background.
  - Automatic Rollback mechanism to revert UI state in case of network or server failure.

Optimizing User Experience (UX)

Live Historical Charting

Recharts: sliding time window (last 20 samples), updating dynamically on every WebSocket event.

Optimistic UI for Actuators

Immediate visual feedback upon operator click. Asynchronous REST API (POST) execution in the background. Automatic Rollback mechanism to revert UI state in case of network or server failure.

Dynamic Automation Engine

- **Dynamic UI Components:** IF/THEN dropdowns automatically populate by extracting keys from the live application state.
- **Future-Proof Scalability:** New backend sensors are instantly integrated into the rules menu without any frontend code changes.
- **Full API Integration:** Complete rule lifecycle management (Initial Fetch, POST for creation, DELETE for removal) fully synced with the PostgreSQL database.

Dynamic Automation Engine

Dynamic UI Components

IF/THEN dropdowns automatically populate by extracting keys from the live application state.

Future-Proof Scalability

New backend sensors are instantly integrated into the rules menu without any frontend code changes.

Full API Integration

Complete rule lifecycle management (Initial Fetch, POST for creation, DELETE for removal) fully synced with the PostgreSQL database.

Tech Stack & Deployment

- **Core:** React.js + Vite (for ultra-fast builds and hot-reloading).
- **Styling:** TailwindCSS (Dark/Sci-Fi design system, responsive, utility-first).
- **Deployment:** Containerized via Docker.
- **Configuration:** Environment variables (VITE_API_URL) injected via docker-compose.yml for "Plug & Play" microservices integration.

Tech Stack & Deployment

Core

React.js + Vite (for ultra-fast builds and hot-reloading).

Styling

TailwindCSS (Dark/Sci-Fi design system, responsive, utility-first).

Deployment

Containerized via Docker.

Configuration

Environment variables (VITE_API_URL) injected via docker-compose.yml for 'Plug & Play' microservices integration.